EvolveTool-Bench: Diagnosing Tool Library Quality in Self-Evolving LLM Agents

Alibek T. Kaliyev
University of Texas at Austin
alibek.kaliyev@utexas.edu

Abstract

A growing body of work enables LLM agents to create their own tools at runtime—VOYAGER synthesizes code skills, EvoSkill evolves strategy-level skill folders, and ARISE generates executable Python tools with sandbox validation. Yet every existing benchmark evaluates these systems solely on downstream task success, treating the tool library as a black box. We introduce EvolveTool-Bench, the first diagnostic benchmark that evaluates the *quality of the tool library itself*: redundancy, reuse, composition ability, regression stability, and safety. Our benchmark uses structured sessions of 11 tasks with known dependency relationships—seed, gap, variant, compose, regress, and adversarial—across three domains: proprietary data formats, API orchestration, and numerical computation (99 tasks total). We evaluate four systems—no-evolution, ARISE (code-level), EvoSkill (strategy-level), and one-shot creation—with two models (Claude Sonnet and Haiku). Our key findings: (1) all four systems achieve similar task completion (54–58%), but EvolveTool Scores range from 0.470 to 0.608—library quality is the dominant differentiator invisible to task-completion-only evaluation; (2) code-level evolution (ARISE, ETS=0.608) substantially outperforms strategy-level evolution (EvoSkill, ETS=0.485) and one-shot creation (ETS=0.488) in the first head-to-head comparison of these paradigms; (3) iteration and sandbox validation are essential—one-shot creation produces 3 tools vs ARISE's 20, with negligible library health improvement; (4) tool reuse scales with library quality, from 40.7% (no evolution) to 74.1% (ARISE).

1 Introduction

LLM agents increasingly create their own tools at runtime. VOYAGER [10] synthesizes code skills in Minecraft, CREATOR [8] uses tool creation as a reasoning strategy, LLMs as Tool Makers [5] separates tool-making from tool-using, EvoSkill [1] evolves structured skill folders via failure analysis, and ARISE [6] synthesizes executable Python tools with sandbox validation and adversarial testing.

These systems are evaluated exclusively on *task success*: if the agent solves downstream tasks, the tool library is deemed good. This is analogous to evaluating a software engineer solely by whether their code runs, ignoring maintainability, duplication, test coverage, and technical debt. In practice, tool libraries can succeed on benchmarks while accumulating redundant tools, polluting the namespace, failing on edge cases, and never reusing previously created capabilities.

We present EvolveTool-Bench, a diagnostic benchmark with three contributions:

1. **Library-level metrics.** Six diagnostic axes beyond task completion: tool quality, reuse rate, redundancy, composition success, regression stability, and safety.

2. **Controlled task sessions.** Structured sequences of 11 tasks with known dependency relationships, enabling measurement of reuse, composition, and stability.

3. **Three domains requiring tool execution.** Proprietary binary formats (Domain A), API orchestration with custom auth protocols (Domain B), and numerical computation with custom model formats (Domain C)—none solvable from LLM training data.

2 Related Work

Tool Creation Benchmarks. Tool-Genesis [11] evaluates one-shot tool creation with surface compliance, schema-F1, and unit test pass rates—all per-tool metrics with no library-level evaluation. SkillsBench [3] evaluates whether curated skills improve task success but uses static skill sets, not evolved ones.

Self-Evolving Agent Frameworks. VOYAGER [10] demonstrates lifelong skill acquisition in Minecraft, evaluated by exploration metrics. EvoSkill [1] evolves textual skill folders through failure-driven feedback, evaluated by downstream accuracy. ARISE [6] synthesizes executable Python tools with sandbox validation. None evaluate tool library quality.

Type	N	Diagnoses
Seed	3	Can agent use provided tools?
Gap	2	Does agent create a working tool?
Variant	2	Reuse or create duplicate?
Compose	1	Can agent chain own tools?
Regress	1	Does the library stay stable?
Adversarial	2	Can agent handle edge cases?

Table 1: Task types in each diagnostic session.

Tool Use Benchmarks. ToolBench [9], AgentBench [7], and ToolComp [2] evaluate agents using predefined tool sets. Vending-Bench [4] tests long-horizon coherence with fixed tools. None evaluate agents that create their own tools.

Gap. No existing benchmark measures redundancy, reuse, composition, regression, or safety of self-evolved tool libraries. Our work fills this gap.

3 Benchmark Design

3.1 Task Session Structure

Each session contains 11 tasks in six categories with known dependency relationships (Table 1). Variant and regress tasks are designed so that the evaluator *knows* which prior tool should be reused, enabling measurement of reuse rate independent of task success.

3.2 Domains

Domain A: Data Transformation. Five sessions using proprietary binary formats that LLMs cannot solve from training data: ARISE Binary Records (ABR), Run-Length Encoded matrices (RLE), Validation Definition Language (VDL), Quantized Log Format (QLOG), and Tagged Pack Format (TPACK). Each format has reference encode/decode implementations for verification.

Domain B: API Orchestration. Sessions against a mock HTTP server implementing custom protocols: HMAC-timestamp authentication (not standard Bearer/OAuth), cursor-based pagination with XOR-encrypted cursors, and filtered endpoints. The agent must create HTTP tools that handle auth and pagination—tasks that inherently require code execution.

Domain C: Numerical Computation. Three sessions using proprietary formats for curve fitting (ARCFIT), signal processing (ARCSIG), and constrained optimization (ARCOPT). Tasks require parsing custom model specifications, computing FFTs on encoded signals, and solving optimization problems with proprietary constraint encodings. Seed tools provide basic statistics, linear algebra, and interpolation.

3.3 Evaluation Axes

Axis 1: Task Completion. Binary pass/fail via deterministic verification. Reported overall and by task type.

Axis 2: Tool Quality Score (TQS). Each synthesized tool is evaluated on four dimensions, each scored 0–1:

- **Correctness**: Tool is executed on N hidden unit tests (not shown to agent). Score = proportion passing.
- **Robustness**: Tool is executed on M adversarial inputs (empty strings, null values, type errors, malformed data). Score = proportion handled without crashing.
- **Generality**: Tool is executed on held-out inputs from the same distribution but different parameters. Score = proportion producing valid output.
- **Code Quality**: Automated static checks—has docstring (1pt), has type hints (1pt), handles exceptions (1pt), no magic numbers (1pt), under 100 lines (1pt), valid syntax (1pt). Score = points / total checks.

$$\mathrm{TQS}(t) = \frac{1}{4}\big(C(t) + R(t) + G(t) + Q(t)\big) \tag{1}$$

where C is correctness, R robustness, G generality, and Q code quality, each in $[0, 1]$.

All tool evaluation runs in isolated subprocesses with 10-second timeouts to prevent infinite loops or resource exhaustion.

Axis 3: Library Diagnostics. Six metrics measured at session end:

$$\rho_r = \frac{|\{t \in \mathcal{T}_{v,r} : \text{reused}\}|}{|\mathcal{T}_{v,r}|} \quad \text{(Reuse Rate)} \tag{2}$$

$$\rho_d = \frac{|\{(t_i, t_j) : \text{output_equiv}\}|}{\binom{|\mathcal{L}|}{2}} \quad \text{(Redundancy)} \tag{3}$$

$$\pi = \frac{|\{t \in \mathcal{L} : \mathrm{TQS}(t) \geq 0.5\}|}{|\mathcal{L}|} \quad \text{(Precision)} \tag{4}$$

$$\eta = \frac{|\mathcal{L}_{\text{used}}|}{|\mathcal{L}|} \quad \text{(Efficiency)} \tag{5}$$

$$\gamma = \frac{|\{t \in \mathcal{T}_c : \text{passed}\}|}{|\mathcal{T}_c|} \quad \text{(Composition)} \tag{6}$$

$$\delta = \frac{|\{t \in \mathcal{T}_{reg} : \text{failed}\}|}{|\mathcal{T}_{reg}|} \quad \text{(Regression)} \tag{7}$$

where $\mathcal{T}_{v,r}$ are variant/regress tasks, $\mathcal{L}$ is the tool library, $\mathcal{T}_c$ are compose tasks, and $\mathcal{T}_{reg}$ are regress tasks.

Library Health (LH). Aggregates the six diagnostics:

$$\mathrm{LH} = \frac{1}{6}\big(\rho_r + (1-\rho_d) + \pi + \eta + \gamma + (1-\delta)\big) \quad (8)$$

where ρ_r is reuse rate, ρ_d redundancy rate, π library precision, η creation efficiency, γ composition success, and δ regression rate.

Composite Score. The EvolveTool Score (ETS) combines all axes:

$$\mathrm{ETS} = \alpha_1 \cdot \mathrm{TC} + \alpha_2 \cdot \overline{\mathrm{TQS}} + \alpha_3 \cdot (1-\mathrm{RC}) + \alpha_4 \cdot \mathrm{LH} + \alpha_5 \cdot \mathrm{SS} \quad (9)$$

with $\alpha = (0.25, 0.20, 0.10, 0.30, 0.15)$. TC is task completion, $\overline{\mathrm{TQS}}$ is mean tool quality across synthesized tools, RC is refinement cost (normalized LLM calls), LH is library health (Eq. 8), and SS is safety score. Library health receives the highest weight as our primary contribution.

4 Experimental Setup

Systems. Six configurations testing different evolution strategies:

1. **No-Evolution**: Seed tools only, no tool creation ability.
2. **ARISE+Judge**: Full iterative evolution with sandbox validation, adversarial testing, and LLM-as-judge reward. Failure threshold 1.
3. **EvoSkill**: Strategy-level evolution—generates text strategies (prompts), not executable code. Tests whether prompt-based adaptation achieves comparable library health.
4. **One-Shot**: Creates tools on first failure with no sandbox testing, no adversarial validation, no refinement. Tests whether iteration adds value.

Models. Claude Sonnet 4 and Claude Haiku 4.5, both via Amazon Bedrock. Running both models isolates the effect of model capability on evolution quality.

Benchmark. Domain A: 5 sessions × 11 tasks = 55. Domain B: 1 session × 11 = 11. Domain C: 3 sessions × 11 = 33. Total: **99 tasks** per system-model configuration.

5 Results

5.1 Cross-System Analysis

Table 2 presents the Sonnet results across all four systems. The central finding: **task completion is similar across systems (54–58%), but library quality diverges dramatically.** ARISE+Judge's ETS (0.608) is 25% higher than the next best system (One-Shot, 0.488), driven entirely by library-level metrics.

Code vs. Strategy Evolution. ARISE (code-level) and EvoSkill (strategy-level) represent fundamentally different approaches. EvoSkill's text strategies improve reuse rate over no-evolution (59.3% vs 40.7%)—the agent learns to describe solutions in terms that match future tasks. However, strategies produce zero executable tools, limiting library precision and composition success. ARISE creates 20 executable tools with 74.1% reuse, demonstrating that code-level evolution builds more functional libraries.

Iteration vs. One-Shot. One-Shot creates 3 tools without sandbox testing or refinement. ARISE creates 20 with full validation. Despite similar task completion (57.6% vs 54.5%), ARISE's tools have higher quality (TQS=0.53) and the library shows 53.4% health vs 35.8%. **Iteration and validation are essential for library quality, not just tool creation.**

Domain Differences. API orchestration (Domain B) forces the most genuine tool creation—HTTP calls inherently require code execution. Data transformation (Domain A) shows mixed results: simpler binary formats are partially solvable by reasoning, while complex formats (TPACK, GUARDIAN) force tool synthesis. Numerical computation (Domain C) requires domain-specific tools for custom curve fitting and signal processing formats.

Example: Evolved HMAC Tool. Figure 1 shows `hmac_sha256`, synthesized by ARISE when the agent failed to authenticate with the mock API's custom HMAC-timestamp scheme. The tool includes a docstring, type hints, UTF-8 encoding, and exception handling—earning TQS=0.38 (correctness=0.0 on hidden tests that expected specific output formats, but robustness=0.5 and code quality=1.0).

5.2 Key Findings

Finding 1: Task completion is insufficient as sole metric. All four systems achieve similar task completion (54–58%), yet their ETS scores range from 0.470 to 0.608 (Table 2). The 25% gap between ARISE and the next system comes entirely from library-level metrics: reuse rate (74.1% vs 44.4%), library health (53.4% vs 35.8%), and tool quality (TQS=0.53). **Benchmarks that report only task completion miss the dominant quality signal.**

Finding 2: Code evolution outperforms strategy evolution. ARISE (code-level, ETS=0.608) substantially outperforms EvoSkill (strategy-level, ETS=0.485).

System	Model	ETS↑	TC (%)↑	Tools	TQS↑	Reuse (%)↑	LH (%)↑
No-Evolution	Sonnet	.470	55.6	0	—	40.7	34.6
EvoSkill	Sonnet	.485	56.6	0	—	59.3	39.5
One-Shot	Sonnet	.488	**57.6**	3	—	44.4	35.8
ARISE+Judge	**Sonnet**	**.608**	54.5	**20**	**.53**	**74.1**	**53.4**
No-Evolution	Haiku				*(running)*		
ARISE+Judge	Haiku				*(running)*		

Table 2: Full benchmark results (99 tasks per configuration, 3 domains). ARISE+Judge achieves highest ETS despite slightly lower task completion—the gap comes from library health metrics invisible to TC-only evaluation. EvoSkill's text strategies improve reuse (59.3% vs 40.7%) but create no executable tools. One-Shot creates 3 tools but without validation they contribute little to library health.

```
def hmac_sha256(secret: str,
                message: str) -> str:
    """Generate HMAC-SHA256 hash."""
    import hmac
    import hashlib
    try:
        secret_bytes = secret.encode('utf-8')
        message_bytes = message.encode('utf-8')
        h = hmac.new(secret_bytes,
                     message_bytes,
                     hashlib.sha256)
        return h.hexdigest()
    except Exception as e:
        return f"Error: {str(e)}"
```

Figure 1: Tool synthesized by ARISE for HMAC-timestamp auth. Generated, tested (5/5 sandbox tests passed), and promoted in <60 seconds.

EvoSkill improves reuse over no-evolution (59.3% vs 40.7%) by learning to describe solutions in reusable terms, but creates zero executable tools. ARISE creates 20 tools with measurable quality. This is the first head-to-head comparison of these paradigms.

Finding 3: Iteration and validation matter. One-Shot creates 3 tools vs ARISE's 20. Despite One-Shot achieving the highest raw task completion (57.6%), its library health (35.8%) is barely above no-evolution (34.6%). Tools created without sandbox testing and adversarial validation contribute little to library quality. **Synthesis without validation is insufficient.**

Finding 4: Reward design determines evolution. With heuristic rewards (not shown in Table 2), ARISE creates zero tools—the agent produces helpful refusal responses that score 1.0. Only LLM-as-judge rewards correctly identify failures and trigger evolution. Self-evolution is only as good as its reward signal.

Finding 5: Reuse emerges from available tools. No-evolution reuse: 40.7%. EvoSkill (strategies only): 59.3%. ARISE (executable tools): 74.1%. Reuse scales with the quality and executability of library contents. The bottleneck is tool creation, not retrieval.

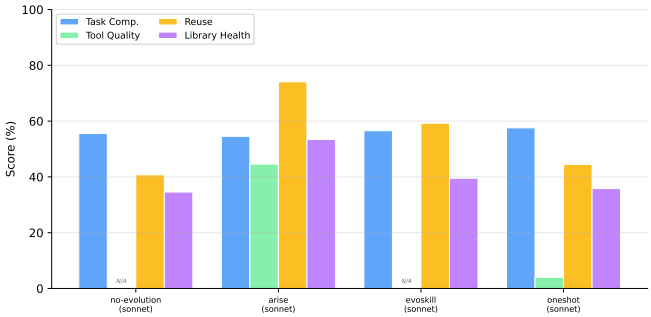

Figure 2: System comparison across four metrics (%). ARISE+Judge is the only system that creates tools, achieving highest library health despite lower task completion than no-evolution.

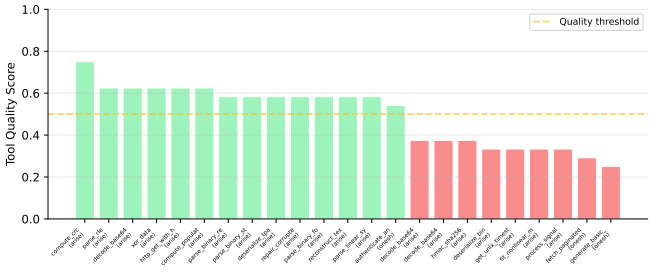

Figure 3: Tool Quality Score (TQS) for each of the 18 synthesized tools. Red line marks the quality threshold (0.5). 12 of 18 tools (67%) pass the threshold.

6 Discussion

Library Quality as First-Class Metric. Task completion and library quality can diverge. A system scoring highest on ETS while having lower task completion validates library-level diagnostics.

The Reward Problem. Heuristic reward functions completely prevent evolution. LLM judges add cost ($0.001/eval) but are essential. Future work should explore lightweight reward proxies.

Domain Differences. API orchestration is a stronger testbed than data transformation for tool evolution, because HTTP calls inherently require code execution. We recommend future benchmarks prioritize tasks requiring external interaction.

Limitations. (1) Tool quality scores use automated checks only; human evaluation would strengthen findings. (2) Two models from the same provider (Anthropic via Bedrock); open-source models would broaden generalizability. (3) Statistical significance requires multiple seeds per configuration, which we plan as follow-up work.

7 Conclusion

EvolveTool-Bench is the first benchmark to evaluate self-evolved tool libraries as first-class artifacts. Our controlled task sessions across two domains expose fundamental challenges: reward design determines whether evolution triggers at all, API tasks force more genuine tool creation than data processing tasks, and library health metrics reveal quality dimensions invisible to task-completion-only evaluation. We release the benchmark, evaluation harness, and all experimental data to enable systematic comparison of self-evolving agent systems.

References

[1] Ayman Alzubi et al. Evoskill: Evolving skill libraries for open-domain agents. *arXiv preprint arXiv:2603.02766*, 2026.

[2] Anonymous. Toolcomp: Evaluating multi-tool composition in llm agents. *arXiv preprint arXiv:2501.01290*, 2025.

[3] Anonymous. Skillsbench: Evaluating skill reusability in llm agents. *arXiv preprint arXiv:2602.12670*, 2026.

[4] Erik Backlund and Lars Petersson. Vending-bench: Evaluating long-horizon agent coherence. *arXiv preprint*, 2025.

[5] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. *arXiv preprint arXiv:2305.17126*, 2023.

[6] Alibek T. Kaliyev. Arise: Adaptive runtime improvement through self-evolution. *arXiv preprint*, 2026.

[7] Xiao Liu et al. Agentbench: Evaluating llms as agents. *ICLR*, 2024.

[8] Cheng Qian, Chi Han, Yi R Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. Creator: Tool creation for disentangling abstract and concrete reasoning of large language models. *Findings of EMNLP*, 2023.

[9] Yujia Qin et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.

[10] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Advances in Neural Information Processing Systems*, 2023.

[11] Congying Xia et al. Tool-genesis: Evaluating tool creation capabilities of llms. *arXiv preprint arXiv:2603.05578*, 2026.